VENU. We built a dynamic-analysis tool, a fuzzer, for PostgreSQL that looks for wrong results, not only crashes, and then generalized it to four more SQL engines. We will spend most of the time on how the tool works and how it found its bugs: eleven PostgreSQL bugs reported, nine fixed upstream, six with our patches; five more bugs reported with fixes to DuckDB, Turso and DataFusion.

VENU. The software-analysis question is the oracle problem. A fuzzer can run millions of queries, but it must decide automatically whether an answer is wrong. Our tool decides by comparison: the same question asked in a different but equivalent way must give the same answer.

VENU. Three parts: the PostgreSQL fuzzer, how its bugs were found, and the generalization.

VENU. Top row: the fuzzing loop. Step 1 derives a test case from the typed grammar as derivation trees. Step 2 rewrites the trees into oracle checks. Step 3 runs everything on master built with assertions, under other planner settings, and on release 18.6. Step 4 classifies, reduces and localizes failures. Bottom row: the LLM loop extends the grammar from recent commits, but only what passes a gate on the real server. Feedback (new plan shapes, new outcomes) goes back into the corpus and the weights.

GOWTHAM. The grammar is our own format. Every expression carries its SQL type, so whatever we generate type-checks, and the server spends its time on planning and executing, not on rejecting syntax. Keeping the tree lets us replace any subtree with another of the same type, which the next three techniques use. Bandit weights raise alternatives that led to new plan shapes; the data profile picks literals near the values in the tables, so predicates select some rows.

BHUVAN. Eight oracles, each catching a different class. The first three only watch the server. The other five compare two answers that must agree. A plan variant changes how PostgreSQL runs a query, never what it returns. Cross-version compares master with the last release. Comparisons are only made where they are sound: we skip non-deterministic queries.

BHUVAN. A string rewriter does not know where a predicate starts or what type an expression has. On the tree both are known, so the rewrite is type-correct at any nesting level. For example, wrapping an expression in a sub-select must not change its value; it does change what the planner can simplify, which is exactly what found one of our bugs.

BHUVAN. Triage turns a raw failure into something a maintainer will read. The tree reduction is the fourth use of the tree: it shrinks by replacing subtrees with minimal ones and replays, so the candidate stays valid SQL and the server's time is spent only on the question whether the failure remains. Localization tells us which planner setting or which rewrite matters, which points at the faulty code.

GOWTHAM. The LLM reads a recent commit and proposes grammar for the feature it touches. We never trust it: a fragment enters the fuzzer only if PostgreSQL accepts what it generates and it actually executes the lines the commit changed, measured with gcov. In our runs 4 of 10 commits passed. Our lesson: a tested rewrite is not a proven one; one accepted rewrite changed a collation.

GOWTHAM. RQ1: typing pays, valid queries reach far more backend code. RQ2: the oracles find real bugs, but false positives were the main cost; we fixed every class we found. RQ3 is our ground truth: we re-injected 23 real bugs, and one hour per trial finds few. RQ4: the LLM adds features, not coverage.

GOWTHAM and BHUVAN share this slide. Read across: each bug was found by a specific technique. Two came only from comparing versions; one only from the sub-select rewrite, which changes what the planner may simplify; one only from the hang oracle; one only from the estimate oracle. That is the point of having a stack of oracles. Every one of them is rediscovered by the tool in one 150-second run.

VENU. Every bug went through the same steps; only the first box differs. The tool does everything up to localization automatically; root cause and patch are our work, and maintainers committed six of our patches.

VENU. The PostgreSQL fuzzer is tied to PostgreSQL in three places: the executor, the grammar and the planner settings. Each became an interface: a driver per engine in its own worker process, a dialect overlay on the same grammar, and optimizer-rule switches. Everything else, the trees, oracles and triage, is reused unchanged.

VENU. Rule firing is our engine-independent feedback: it needs no instrumented build and it is about the optimizer, where logic bugs live. For SQLite we call a test-control function that the Python binding does not export, by finding its address in the symbol table.

BHUVAN. Data synthesis attacks vacuous checks: a TLP check proves nothing if the table has no rows for some partition. Normalizers make two different engines comparable without hand-written rules. The hunt loop runs the whole pipeline without us and drafts the package we file.

GOWTHAM. Same pipeline as for PostgreSQL. The Turso bug was found only by comparing with SQLite; the others by the internal-error oracle. Before filing, we re-checked each one on the engine's newest code built from source; three had already been fixed, which we found by bisection. Five are filed with a fix; the DuckDB one is approved by a maintainer, the others are in review. One script reproduces all five on the released versions.

EACH MEMBER says one sentence about their part.

VENU. We state plainly what the results do not show.

VENU. Close with the numbers and the take-away.